

TaskFlow Pro — Known Failure Cases

Contata Hackathon 2026 — Testing & Reliability

Why we're listing these

Every one of these is something we noticed, thought about, and made a call on — usually 'fix it now' or 'note it and move on,' given the time we had. None of it was found by an outside reviewer or reported after the fact; we found all of it ourselves while testing, mostly by trying to break our own app on purpose in the last stretch before submission.

We're separating this from the architecture document's Known Limitations section on purpose. That section is about scope we deliberately left out (working days, multi-tenancy, and so on). This one is about specific ways the system can behave unexpectedly within the scope we did build — smaller, sharper, and more useful for anyone trying to poke holes in it.

Failure cases

Scenario	What happens	Severity	Would fix by
Two users edit the same graph at the same time	Last write wins — the second save can silently overwrite the first person's change	Medium	Add optimistic locking (a version column) so the second write fails loudly instead of quietly
Very large graphs (several thousand tasks)	Critical path is recalculated in full each time rather than incrementally, so it gets slower as the graph grows	Low for this scope	Cache the critical path and only recompute the affected portion
AI suggestion request during an API outage	The suggest-dependencies call times out after ~15s with a generic error toast; it doesn't yet distinguish 'no key configured' from 'Anthropic is down' from 'rate limited'	Low	Return a specific error code per failure type so the UI can message each one differently
Deleting a task mid-drag	If a task is deleted from another tab while someone is dragging it on the board, the drag can complete against a task that no longer exists, and the UI has to recover on the next refresh rather than in place	Low	Add an optimistic-lock check on the move endpoint, same fix as the concurrent-edit case above
Extremely long dependency chains (50+ deep)	Works correctly in testing, but the schedule-impact panel lists every affected task with no grouping, so the panel gets long and a bit overwhelming to read	Cosmetic	Collapse the impact list beyond ~10 tasks with a 'show all' toggle
Non-ASCII / very long task titles sent to the AI suggester	Not fully tested — we validated the suggestion pipeline mostly with English, moderate-length titles	Unknown	Add explicit test cases before relying on this for non-English content
Browser refresh mid-drag	The drag is abandoned and the card returns to its last saved position — correct behaviour, but there's no toast confirming that's what happened, so it can look like nothing happened at first glance	Cosmetic	Add a brief toast on reconnect explaining the in-flight drag was cancelled

The one we're least comfortable with

The concurrent-edit case (top of the table) is the one that would actually bite a real team. A hackathon demo is one person clicking around, so we never hit it ourselves during testing — we found it by reasoning through the code, not by reproducing it live. If we'd had another day, that's where it would have gone. We'd rather say that plainly than have it look like we didn't think about it.

What held up well

The engine itself — cycle detection, the diamond no-compounding case, rollback on regression — didn't produce a single failure case worth listing here once the test suite (see our separate Test Cases document) was passing. That's the part we spent the most hours hardening early on, and it shows. Most of what's above is at the edges: concurrency, scale, and error messaging, rather than the core scheduling logic being wrong.